Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 8: Encapsulation & Static (OOP-2)

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. How Objects Access Attributes
2. Attribute Creation from Outside the Class
3. Reference Variables
4. Pass by Reference (Objects in Functions)
5. Mutability of Objects
6. Encapsulation
7. Collection of Objects
8. Static (Class) Variables
9. Static Methods

54 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 8: Encapsulation & Static (OOP-2) - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: How Objects Access Attributes, Attribute Creation from Outside the Class, Reference Variables, Pass by Reference (Objects in Functions), Mutability of Objects, Encapsulation, Collection of Objects, Static (Class) Variables, Static Methods
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. How Objects Access Attributes

Q1. Dot Notation

BEGINNER

Create class Laptop(brand, ram). Create an object and read both attributes with dot notation.

Solution:

```
class Laptop:
    def __init__(self, brand, ram):
        self.brand = brand
        self.ram = ram

l = Laptop("Dell", 16)
print(l.brand)
print(l.ram)
```

Output:

```
Dell
16
```

Q2. Missing Attribute

BEGINNER

Try to read l.price (not defined) inside try-except and print the AttributeError.

Solution:

```
class Laptop:
    def __init__(self, brand):
        self.brand = brand

l = Laptop("HP")
try:
    print(l.price)
except AttributeError as e:
    print(e)
```

Output:

```
'Laptop' object has no attribute 'price'
```

Q3. getattr() and hasattr()

INTERMEDIATE

Use hasattr() to check if a laptop has "ram" and "price". Use getattr() with a default value for "price".

Solution:

```
class Laptop:
    def __init__(self, brand, ram):
        self.brand = brand
        self.ram = ram

l = Laptop("Lenovo", 8)
print(hasattr(l, "ram"))
print(hasattr(l, "price"))
print(getattr(l, "price", "Not set"))
```

Output:

```
True
False
Not set
```

Q4. View All Attributes with __dict__

INTERMEDIATE

Print the __dict__ of a Student object with name and city to see all instance attributes as a dictionary.

Solution:

```
class Student:
    def __init__(self, name, city):
        self.name = name
        self.city = city

s = Student("Om", "Pune")
print(s.__dict__)
```

Output:

```
{'name': 'Om', 'city': 'Pune'}
```

Q5. Instance First, Then Class

ADVANCED

Class Shop has class attribute city = "Pune". Object s2 sets its own city = "Goa". Print city for both objects and the class to see the lookup order.

Solution:

```
class Shop:
    city = "Pune"

s1 = Shop()
s2 = Shop()
s2.city = "Goa"

print(s1.city)
print(s2.city)
print(Shop.city)
print(s2.__dict__)
```

Output:

```
Pune
Goa
Pune
{'city': 'Goa'}
```

Q6. Read Attributes Dynamically

ADVANCED

Given a list of field names ["name", "dept", "salary"], print each field of an Employee object using getattr() in a loop.

Solution:

```
class Employee:
    def __init__(self, name, dept, salary):
        self.name = name
        self.dept = dept
        self.salary = salary

e = Employee("Ajay", "IT", 61000)
for field in ["name", "dept", "salary"]:
    print(field, getattr(e, field))
```

Output:

```
name Ajay
dept IT
salary 61000
```

2. Attribute Creation from Outside the Class

Q7. Add an Attribute Later

BEGINNER

Create class Car(brand). After creating an object, add color = "Red" from outside and print it.

Solution:

```
class Car:
    def __init__(self, brand):
        self.brand = brand

c = Car("Maruti")
c.color = "Red"
print(c.brand, c.color)
```

Output:

```
Maruti Red
```

Q8. Only That Object Gets It

BEGINNER

Add color to c1 only. Check hasattr() for c1 and c2.

Solution:

```
class Car:
    def __init__(self, brand):
        self.brand = brand

c1 = Car("Tata")
c2 = Car("Kia")
c1.color = "Blue"

print(hasattr(c1, "color"))
print(hasattr(c2, "color"))
```

Output:

```
True
False
```

Q9. setattr() and delattr()

INTERMEDIATE

Use setattr() to add "discount" = 10 to a Product object and then delattr() to remove it. Print __dict__ after each step.

Solution:

```
class Product:
    def __init__(self, name):
        self.name = name

p = Product("Bag")
setattr(p, "discount", 10)
print(p.__dict__)

delattr(p, "discount")
print(p.__dict__)
```

Output:

```
{'name': 'Bag', 'discount': 10}
{'name': 'Bag'}
```

Q10. Typo Creates a New Attribute

INTERMEDIATE

An Account has balance. By mistake the code writes acc.balnce = 900. Show that Python silently creates a new attribute and balance is unchanged.

Solution:

```
class Account:
    def __init__(self):
        self.balance = 500

acc = Account()
acc.balnce = 900          # typo!
print(acc.balance)
print(acc.__dict__)
```

Output:

```
500
{'balance': 500, 'balnce': 900}
```

Q11. Stop Outside Attributes with __slots__

ADVANCED

Create class Point with __slots__ = ("x", "y"). Try to add z from outside inside try-except.

Solution:

```
class Point:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
try:
    p.z = 3
except AttributeError as e:
    print(e)
```

Output:

```
'Point' object has no attribute 'z' and no __dict__ for setting
new attributes
```

Q12. Build Object from a Dictionary

ADVANCED

Create an empty class Config and set attributes from the dictionary {"host": "localhost", "port": 5432} using setattr in a loop.

Solution:

```
class Config:
    pass

settings = {"host": "localhost", "port": 5432}
cfg = Config()
for key, value in settings.items():
    setattr(cfg, key, value)

print(cfg.host)
print(cfg.port)
```

Output:

```
localhost
5432
```

3. Reference Variables

Q13. Two Names, One Object

BEGINNER

Create a Phone object p1 and set p2 = p1. Change p2.model and print p1.model.

Solution:

```
class Phone:
    def __init__(self, model):
        self.model = model

p1 = Phone("Galaxy")
p2 = p1
p2.model = "iPhone"
print(p1.model)
print(p1 is p2)
```

Output:

```
iPhone
True
```

Q14. Two Separate Objects

BEGINNER

Create p1 = Phone("A") and p2 = Phone("A"). Check p1 is p2 and compare their model values.

Solution:

```
class Phone:
    def __init__(self, model):
        self.model = model

p1 = Phone("A")
p2 = Phone("A")
print(p1 is p2)
print(p1.model == p2.model)
```

Output:

```
False
True
```

Q15. Object Without a Reference

INTERMEDIATE

Call Phone("Temp").model directly without storing the object in a variable. Explain in a comment what happens to the object.

Solution:

```
class Phone:
    def __init__(self, model):
        self.model = model

print(Phone("Temp").model)
# No variable refers to this object, so Python's garbage
# collector removes it after this line.
```

Output:

```
Temp
```

Q16. Reference Count

INTERMEDIATE

Use sys.getrefcount() to see the reference count of an object increase when new variables point to it.

Solution:

```
import sys

class Item:
    pass

a = Item()
before = sys.getrefcount(a)
b = a
c = a
after = sys.getrefcount(a)
print(after - before)
```

Output:

```
2
```

Q17. Reassigning a Reference

ADVANCED

Create p = Phone("Old"), q = p, then p = Phone("New"). Print p.model and q.model and explain.

Solution:

```
class Phone:
    def __init__(self, model):
        self.model = model

p = Phone("Old")
q = p
p = Phone("New")
print(p.model)
print(q.model)
# p now points to a new object; q still points to the old one
```

Output:

```
New
Old
```

Q18. List of References

ADVANCED

Create one Counter object and put the SAME object 3 times in a list. Increase the count through list[0] and print through list[2].

Solution:

```
class Counter:
    def __init__(self):
        self.count = 0

c = Counter()
items = [c, c, c]
items[0].count += 5
print(items[2].count)
print(items[0] is items[2])
```

Output:

```
5
True
```

4. Pass by Reference (Objects in Functions)

Q19. Function Changes the Object

BEGINNER

Write give_bonus(emp) that adds 5000 to emp.salary. Show that the original object changes.

Solution:

```
class Employee:
    def __init__(self, salary):
        self.salary = salary

def give_bonus(emp):
    emp.salary += 5000

e = Employee(40000)
give_bonus(e)
print(e.salary)
```

Output:

```
45000
```

Q20. Function Returns a New Object

BEGINNER

Write make_customer(name) that creates and returns a Customer object. Print its name.

Solution:

```
class Customer:
    def __init__(self, name):
        self.name = name

def make_customer(name):
    return Customer(name)

c = make_customer("Riya")
print(c.name)
```

Output:

```
Riya
```

Q21. Same Object Inside and Outside

INTERMEDIATE

Print id() of an object outside and inside a function to prove the same object is passed.

Solution:

```
class Box:
    pass

def check(obj):
    return id(obj)

b = Box()
print(check(b) == id(b))
```

Output:

```
True
```

Q22. Reassigning Inside Function

INTERMEDIATE

Write reset(emp) that does emp = Employee(0). Show that the original object outside is NOT changed.

Solution:

```
class Employee:
    def __init__(self, salary):
        self.salary = salary

def reset(emp):
    emp = Employee(0)

e = Employee(40000)
reset(e)
print(e.salary)
```

Output:

```
40000
```

Q23. Mutable vs Immutable Arguments

ADVANCED

Write update(n, lst) that does n += 1 and lst.append(1). Call it with x = 5 and nums = [0] and print both after the call.

Solution:

```
def update(n, lst):
    n += 1
    lst.append(1)

x = 5
nums = [0]
update(x, nums)
print(x)
print(nums)
```

Output:

```
5
[0, 1]
```

Q24. Avoid Side Effects with copy

ADVANCED

Write discounted(product) that returns a NEW product with 10% off without changing the original. Use copy.copy().

Solution:

```
import copy

class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

def discounted(product):
    new_p = copy.copy(product)
    new_p.price = product.price * 0.9
    return new_p

p = Product("Shoes", 2000)
d = discounted(p)
print(p.price)
print(d.price)
```

Output:

```
2000
1800.0
```

5. Mutability of Objects

Q25. Objects are Mutable

BEGINNER

Create a Student object with marks 60 and change it to 75. Show the id() stays the same.

Solution:

```
class Student:
    def __init__(self, marks):
        self.marks = marks

s = Student(60)
old_id = id(s)
s.marks = 75
print(s.marks)
print(id(s) == old_id)
```

Output:

```
75
True
```

Q26. Strings Inside are Immutable

BEGINNER

Show that changing s.name = s.name + " K" creates a new string object for the attribute (compare ids of the attribute).

Solution:

```
class Student:
    def __init__(self, name):
        self.name = name

s = Student("Om")
before = id(s.name)
s.name = s.name + " K"
print(s.name)
print(id(s.name) == before)
```

Output:

```
Om K
False
```

Q27. Shared List Attribute Problem

INTERMEDIATE

Create class Team with a class variable members = []. Add a member to team1 and see it appear in team2. Explain why.

Solution:

```
class Team:
    members = []          # shared by ALL objects

t1 = Team()
t2 = Team()
t1.members.append("Om")
print(t2.members)
```

Output:

```
['Om']
```

Q28. Fix: Instance List

INTERMEDIATE

Fix the Team class so each team has its own members list created in the constructor.

Solution:

```
class Team:
    def __init__(self):
        self.members = []

t1 = Team()
t2 = Team()
t1.members.append("Om")
print(t1.members)
print(t2.members)
```

Output:

```
['Om']
[]
```

Q29. Shallow vs Deep Copy

ADVANCED

A Classroom has a list of students. Make a shallow copy and a deep copy, add a student to the original and see which copy changes.

Solution:

```
import copy

class Classroom:
    def __init__(self, students):
        self.students = students

c1 = Classroom(["Om", "Pia"])
shallow = copy.copy(c1)
deep = copy.deepcopy(c1)

c1.students.append("Raj")
print(shallow.students)
print(deep.students)
```

Output:

```
['Om', 'Pia', 'Raj']
['Om', 'Pia']
```

Q30. Immutable Object with a Frozen Dataclass

ADVANCED

Create a frozen dataclass Coordinate(lat, lon). Try to change lat inside try-except.

Solution:

```
from dataclasses import dataclass, FrozenInstanceError

@dataclass(frozen=True)
class Coordinate:
    lat: float
    lon: float

c = Coordinate(18.52, 73.85)
print(c)
try:
    c.lat = 19.0
except FrozenInstanceError as e:
    print("Cannot change:", e)
```

Output:

```
Coordinate(lat=18.52, lon=73.85)
Cannot change: cannot assign to field 'lat'
```

6. Encapsulation

Q31. Private Attribute

BEGINNER

Create class Account with a private attribute __balance = 1000. Try to print acc.__balance inside try-except.

Solution:

```
class Account:
    def __init__(self):
        self.__balance = 1000

acc = Account()
try:
    print(acc.__balance)
except AttributeError as e:
    print(e)
```

Output:

```
'Account' object has no attribute '__balance'
```

Q32. Getter Method

BEGINNER

Add a method get_balance() to read the private balance.

Solution:

```
class Account:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

acc = Account(2500)
print(acc.get_balance())
```

Output:

```
2500
```

Q33. Setter with Validation

INTERMEDIATE

Add set_balance(amount) that only accepts non-negative values, otherwise prints "Invalid amount".

Solution:

```
class Account:
    def __init__(self):
        self.__balance = 0

    def set_balance(self, amount):
        if amount < 0:
            print("Invalid amount")
        else:
            self.__balance = amount

    def get_balance(self):
        return self.__balance

acc = Account()
acc.set_balance(3000)
acc.set_balance(-50)
print(acc.get_balance())
```

Output:

```
Invalid amount
3000
```

Q34. Name Mangling

INTERMEDIATE

Show that a private attribute is stored as _ClassName__attr by printing __dict__ and reading it with the mangled name (not recommended in real code).

Solution:

```
class Account:
    def __init__(self):
        self.__pin = 4321

acc = Account()
print(acc.__dict__)
print(acc._Account__pin)
```

Output:

```
{'_Account__pin': 4321}
4321
```

Q35. Property Decorator

ADVANCED

Use @property and @salary.setter in class Employee so salary is read like an attribute but cannot be set below 10000.

Solution:

```
class Employee:
    def __init__(self, salary):
        self._salary = salary

    @property
    def salary(self):
        return self._salary

    @salary.setter
    def salary(self, value):
        if value < 10000:
            print("Salary too low")
        else:
            self._salary = value

e = Employee(30000)
e.salary = 5000
print(e.salary)
e.salary = 45000
print(e.salary)
```

Output:

```
Salary too low
30000
45000
```

Q36. ATM with Encapsulation

ADVANCED

Create class ATM with private __pin and __balance. withdraw(pin, amount) checks the PIN and balance before reducing the balance.

Solution:

```
class ATM:
    def __init__(self, pin, balance):
        self.__pin = pin
        self.__balance = balance

    def withdraw(self, pin, amount):
        if pin != self.__pin:
            return "Wrong PIN"
        if amount > self.__balance:
            return "Insufficient balance"
        self.__balance -= amount
        return "Collect Rs." + str(amount)

atm = ATM(1234, 5000)
print(atm.withdraw(1111, 500))
print(atm.withdraw(1234, 8000))
print(atm.withdraw(1234, 2000))
```

Output:

```
Wrong PIN
Insufficient balance
Collect Rs.2000
```

7. Collection of Objects

Q37. List of Students

BEGINNER

Create 3 Student objects (name, marks) in a list and print each name and marks using a loop.

Solution:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

students = [Student("Om", 78),
            Student("Pia", 92),
            Student("Raj", 65)]
for s in students:
    print(s.name, s.marks)
```

Output:

```
Om 78
Pia 92
Raj 65
```

Q38. Total of All Prices

BEGINNER

Create a list of Product objects and find the total price using a loop.

Solution:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

cart = [Product("Pen", 10),
        Product("Book", 60),
        Product("Bag", 450)]
total = 0
for p in cart:
    total += p.price
print(total)
```

Output:

```
520
```

Q39. Find the Topper

INTERMEDIATE

From a list of Student objects find the student with the highest marks using max() with a lambda key.

Solution:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

students = [Student("Om", 78),
            Student("Pia", 92),
            Student("Raj", 65)]
topper = max(students, key=lambda s: s.marks)
print(topper.name)
```

Output:

```
Pia
```

Q40. Dictionary of Objects

INTERMEDIATE

Store Employee objects in a dictionary keyed by employee ID. Look up E102 and print the name.

Solution:

```
class Employee:
    def __init__(self, name):
        self.name = name

staff = {"E101": Employee("Ajay"), "E102": Employee("Seema")}
print(staff["E102"].name)
```

Output:

```
Seema
```

Q41. Filter and Sort Objects

ADVANCED

From a list of Product objects, keep products under Rs. 500 and sort them by price.

Solution:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

items = [Product("Watch", 1500), Product("Cap", 299),
         Product("Belt", 450), Product("Socks", 150)]

under_500 = [p for p in items if p.price < 500]
cheap = sorted(under_500, key=lambda p: p.price)
for p in cheap:
    print(p.name, p.price)
```

Output:

```
Socks 150
Cap 299
Belt 450
```

Q42. Library Management

ADVANCED

Create class Library that holds a list of Book objects with add_book(), and available() returning titles of books not issued.

Solution:

```
class Book:
    def __init__(self, title):
        self.title = title
        self.issued = False

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def available(self):
        return [b.title for b in self.books if not b.issued]

lib = Library()
lib.add_book(Book("Python"))
lib.add_book(Book("SQL"))
lib.add_book(Book("Excel"))
lib.books[1].issued = True
print(lib.available())
```

Output:

```
['Python', 'Excel']
```

8. Static (Class) Variables

Q43. Shared School Name

BEGINNER

Create class Student with a class variable school = "Bikkad IT". Print it from two objects and from the class.

Solution:

```
class Student:
    school = "Bikkad IT"

    def __init__(self, name):
        self.name = name

s1 = Student("Om")
s2 = Student("Pia")
print(s1.school)
print(s2.school)
print(Student.school)
```

Output:

```
Bikkad IT
Bikkad IT
Bikkad IT
```

Q44. Change for Everyone

BEGINNER

Change Student.school to "Bikkad IT Pune" through the class and print it for both objects.

Solution:

```
class Student:
    school = "Bikkad IT"

s1 = Student()
s2 = Student()
Student.school = "Bikkad IT Pune"
print(s1.school)
print(s2.school)
```

Output:

```
Bikkad IT Pune
Bikkad IT Pune
```

Q45. Count Objects Created

INTERMEDIATE

Use a class variable count in class Visitor that increases in the constructor. Create 4 visitors and print the count.

Solution:

```
class Visitor:
    count = 0

    def __init__(self):
        Visitor.count += 1

for i in range(4):
    Visitor()

print(Visitor.count)
```

Output:

```
4
```

Q46. Class Variable vs Instance Variable

INTERMEDIATE

Class Car has wheels = 4 (class variable). Set bike-like car.wheels = 3 on one object only. Print both objects and the class.

Solution:

```
class Car:
    wheels = 4

c1 = Car()
c2 = Car()
c2.wheels = 3          # creates an instance variable on c2 only
print(c1.wheels)
print(c2.wheels)
print(Car.wheels)
```

Output:

```
4
3
4
```

Q47. Common Interest Rate

ADVANCED

Class SavingsAccount has class variable rate = 4.0. Write method interest() using SavingsAccount.rate. Change the rate once and see all accounts update.

Solution:

```
class SavingsAccount:
    rate = 4.0

    def __init__(self, balance):
        self.balance = balance

    def interest(self):
        return self.balance * SavingsAccount.rate / 100

a1 = SavingsAccount(10000)
a2 = SavingsAccount(50000)
print(a1.interest(), a2.interest())

SavingsAccount.rate = 5.0
print(a1.interest(), a2.interest())
```

Output:

```
400.0 2000.0
500.0 2500.0
```

Q48. Unique Invoice Numbers

ADVANCED

Use a class variable next_no = 1001 to give every Invoice a unique number and also keep a class-level list of all invoice numbers.

Solution:

```
class Invoice:
    next_no = 1001
    all_numbers = []

    def __init__(self, amount):
        self.number = Invoice.next_no
        self.amount = amount
        Invoice.all_numbers.append(self.number)
        Invoice.next_no += 1

Invoice(500)
Invoice(1200)
last = Invoice(300)
print(last.number)
print(Invoice.all_numbers)
```

Output:

```
1003
[1001, 1002, 1003]
```

9. Static Methods

Q49. Utility Method

BEGINNER

Create class MathTool with a @staticmethod add(a, b). Call it using the class without creating an object.

Solution:

```
class MathTool:
    @staticmethod
    def add(a, b):
        return a + b

print(MathTool.add(12, 30))
```

Output:

```
42
```

Q50. Temperature Converter

BEGINNER

Create class Converter with static methods c_to_f(c) and f_to_c(f).

Solution:

```
class Converter:
    @staticmethod
    def c_to_f(c):
        return c * 9 / 5 + 32

    @staticmethod
    def f_to_c(f):
        return round((f - 32) * 5 / 9, 2)

print(Converter.c_to_f(37))
print(Converter.f_to_c(100))
```

Output:

```
98.6
37.78
```

Q51. Validation Helper

INTERMEDIATE

Create class Validator with static methods is_valid_mobile(num) and is_valid_pin(pin). Test them.

Solution:

```
class Validator:
    @staticmethod
    def is_valid_mobile(num):
        return (len(num) == 10 and num.isdigit()
                and num[0] in "6789")

    @staticmethod
    def is_valid_pin(pin):
        return len(pin) == 6 and pin.isdigit()

print(Validator.is_valid_mobile("9823456710"))
print(Validator.is_valid_mobile("1234"))
print(Validator.is_valid_pin("411018"))
```

Output:

```
True
False
True
```

Q52. Static Method Used Inside the Class

INTERMEDIATE

Class User calls a static method is_strong(password) from its constructor to set self.strong.

Solution:

```
class User:
    def __init__(self, name, password):
        self.name = name
        self.strong = User.is_strong(password)

    @staticmethod
    def is_strong(password):
        has_digit = any(c.isdigit() for c in password)
        return len(password) >= 8 and has_digit

print(User("Om", "pune2026").strong)
print(User("Pia", "abc").strong)
```

Output:

```
True
False
```

Q53. Static Method vs Class Method

ADVANCED

Create class Pizza with a class variable base_price = 200, a @classmethod margherita() that returns a Pizza object, and a @staticmethod gst(amount).

Solution:

```
class Pizza:
    base_price = 200

    def __init__(self, name, price):
        self.name = name
        self.price = price

    @classmethod
    def margherita(cls):
        return cls("Margherita", cls.base_price)

    @staticmethod
    def gst(amount):
        return amount * 5 / 100

p = Pizza.margherita()
print(p.name, p.price)
print(Pizza.gst(p.price))
```

Output:

```
Margherita 200
10.0
```

Q54. Date Helper Class

ADVANCED

Create class DateTool with a static method is_leap(year) and a static method days_in_month(month, year) that uses is_leap for February.

Solution:

```
class DateTool:
    @staticmethod
    def is_leap(year):
        return ((year % 4 == 0 and year % 100 != 0)
                or year % 400 == 0)

    @staticmethod
    def days_in_month(month, year):
        if month == 2:
            return 29 if DateTool.is_leap(year) else 28
        if month in (4, 6, 9, 11):
            return 30
        return 31

print(DateTool.days_in_month(2, 2024))
print(DateTool.days_in_month(2, 2026))
print(DateTool.days_in_month(11, 2026))
```

Output:

```
29
28
30
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026